Mini RFC: Packet Protection

April 21, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

1 Overview

As with TLS over TCP, QUIC protects packets with keys derived from the TLS handshake, using the AEAD algorithm [RFC 5116] negotiated by TLS.

QUIC packets have varying protections depending on their type:

- Version Negotiation packets have no cryptographic protection.
- `Retry` packets use `AEAD_AES_128_GCM` to provide protection against accidental modification and to limit the entities that can produce a valid Retry.
- `Initial` packets use `AEAD_AES_128_GCM` with keys derived from the Destination Connection ID field of the first Initial packet sent by the client.
- All other packets have strong cryptographic protections for confidentiality and integrity, using keys and algorithms negotiated by TLS.

This document describes how packet protection is applied to `Handshake` packets, `0-RTT` packets, and `1-RTT` packets. The same packet protection process is applied to `Initial` packets. However, as it is trivial to determine the keys used for `Initial` packets, these packets are not considered to have confidentiality or integrity protection. Retry packets use a fixed key and so similarly lack confidentiality and integrity protection.

2 Packet Protection Keys

QUIC derives packet protection keys in the same way that TLS derives record protection keys.

Each encryption level has separate secret values for protection of packets sent in each direction. These traffic secrets are derived by TLS and are used by QUIC for all encryption levels except the Initial encryption level. The secrets for the `Initial` encryption level are computed based on the client's initial Destination Connection ID.

The keys used for packet protection are computed from the TLS secrets using the KDF provided by TLS. In TLS 1.3, the HKDF-Expand-Label function described in Section 7.1 of [RFC 8446] is used with the hash function from the negotiated cipher suite. All uses of HKDF-Expand-Label in QUIC use a zero-length Context.

Note that labels, which are described using strings, are encoded as bytes using ASCII without quotes or any trailing NUL byte.

Other versions of TLS MUST provide a similar function in order to be used with QUIC.

The current encryption level secret and the label "quic key" are input to the KDF to produce the AEAD key; the label "quic iv" is used to derive the Initialization Vector (IV). The header protection key uses the "quic hp" label. Using these labels provides key separation between QUIC and TLS.

Both "quic key" and "quic hp" are used to produce keys, so the Length provided to HKDF-Expand-Label along with these labels is determined by the size of keys in the AEAD or header protection algorithm. The Length provided with "quic iv" is the minimum length of the AEAD nonce or 8 bytes if that is larger.

The KDF used for initial secrets is always the HKDF-Expand-Label function from TLS 1.3.

3 Initial Secret

Initial packets apply the packet protection process, but use a secret derived from the Destination Connection ID field from the client's first Initial packet.

This secret is determined by using HKDF-Extract (see Section 2.2 of [RFC 5869]) with a salt of `0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a` and the input keying material (IKM) of the Destination Connection ID field. This produces an intermediate pseudorandom key (PRK) that is used to derive two separate secrets for sending and receiving.

The secret used by clients to construct Initial packets uses the PRK and the label "client in" as input to the HKDF-Expand-Label function from TLS [RFC 8446] to produce a 32-byte secret. Packets constructed by the server use the same process with the label "server in". The hash function for HKDF when deriving initial secrets and keys is SHA-256.

This process in pseudocode is:

```
initial_salt = 0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a
initial_secret = HKDF-Extract(initial_salt,
                              client_dst_connection_id)

client_initial_secret = HKDF-Expand-Label(initial_secret,
                                          "client in", "",
                                          Hash.length)
server_initial_secret = HKDF-Expand-Label(initial_secret,
                                          "server in", "",
                                          Hash.length)
```

The connection ID used with HKDF-Expand-Label is the Destination Connection ID in the Initial packet sent by the client. This will be a randomly selected value unless the client creates the Initial packet after receiving a Retry packet, where the Destination Connection ID is selected by the server.

The HKDF-Expand-Label function defined in TLS 1.3 MUST be used for `Initial` packets even where the TLS versions offered do not include TLS 1.3.

The secrets used for constructing subsequent `Initial` packets change when a server sends a `Retry` packet to use the connection ID value selected by the server. The secrets do not change when a client changes the Destination Connection ID it uses in response to an `Initial` packet from the server.

4 AEAD Usage

The Authenticated Encryption with Associated Data (AEAD) function (see [RFC 5116]) used for QUIC packet protection is the AEAD that is negotiated for use with the TLS connection. For example, if TLS is using the `TLS_AES_128_GCM_SHA256` cipher suite, the `AEAD_AES_128_GCM` function is used.

QUIC can use any of the cipher suites defined in with the exception of `TLS_AES_128_CCM_8_SHA256`. A cipher suite MUST NOT be negotiated unless a header protection scheme is defined for the cipher suite. This document defines a header protection scheme for all cipher suites defined in aside from `TLS_AES_128_CCM_8_SHA256`. These cipher suites have a 16-byte authentication tag and produce an output 16 bytes larger than their input.

An endpoint MUST NOT reject a ClientHello that offers a cipher suite that it does not support, or it would be impossible to deploy a new cipher suite. This also applies to `TLS_AES_128_CCM_8_SHA256`.

When constructing packets, the AEAD function is applied prior to applying header protection. The unprotected packet header is part of the associated data (A). When processing packets, an endpoint first removes the header protection.

The key and IV for the packet are computed as described above. The nonce, N, is formed by combining the packet protection IV with the packet number. The 62 bits of the reconstructed QUIC packet number in network byte order are left-padded with zeros to the size of the IV. The exclusive OR of the padded packet number and the IV forms the AEAD nonce.

The associated data, A, for the AEAD is the contents of the QUIC header, starting from the first byte of either the short or long header, up to and including the unprotected packet number.

The input plaintext, P, for the AEAD is the payload of the QUIC packet.

The output ciphertext, C, of the AEAD is transmitted in place of P.

Some AEAD functions have limits for how many packets can be encrypted under the same key and IV. This might be lower than the packet number limit. An endpoint MUST initiate a key update prior to exceeding any limit set for the AEAD that is in use.

5 Header Protection

Parts of QUIC packet headers, in particular the Packet Number field, are protected using a key that is derived separately from the packet protection key and IV. The key derived using the "quic hp" label is used to provide confidentiality protection for those fields that are not exposed to on-path elements.

This protection applies to the least significant bits of the first byte, plus the Packet Number field. The four least significant bits of the first byte are protected for packets with long headers; the five least significant bits of the first byte are protected for packets with short headers. For both header forms, this covers the reserved bits and the Packet Number Length field; the Key Phase bit is also protected for packets with a short header.

The same header protection key is used for the duration of the connection, with the value not changing after a key update. This allows header protection to be used to protect the key phase.

This process does not apply to Retry or Version Negotiation packets, which do not contain a protected payload or any of the fields that are protected by this process.

5.1 Header Protection Application

Header protection is applied after packet protection is applied. The ciphertext of the packet is sampled and used as input to an encryption algorithm. The algorithm used depends on the negotiated AEAD.

The output of this algorithm is a 5-byte mask that is applied to the protected header fields using exclusive OR. The least significant bits of the first byte of the packet are masked by the least significant bits of the first mask byte, and the packet number is masked with the remaining bytes. Any unused bytes of mask that might result from a shorter packet number encoding are unused.

Following shows a sample algorithm for applying header protection. Removing header protection only differs in the order in which the packet number length (pn_length) is determined.

```
mask = header_protection(hp_key, sample)

pn_length = (packet[0] & 0x03) + 1
if (packet[0] & 0x80) == 0x80:
    # Long header: 4 bits masked
    packet[0] ^= mask[0] & 0x0f
else:
    # Short header: 5 bits masked
    packet[0] ^= mask[0] & 0x1f

# pn_offset is the start of the Packet Number field.
packet[pn_offset:pn_offset+pn_length] ^= mask[1:1+pn_length]
```

Specific header protection functions are defined based on the selected cipher suite.

Figure 1 and Figure 2 shows an example long header packet (Initial) and a short header packet (1-RTT). They show the fields in each header that are covered by header protection and the portion of the protected packet payload that is sampled.

Before a TLS cipher suite can be used with QUIC, a header protection algorithm MUST be specified for the AEAD used with that cipher suite. This document defines algorithms for AEAD_AES_128_GCM, AEAD_AES_128_CCM, AEAD_AES_256_GCM (all these AES AEADs are defined in [AEAD]), and AEAD_CHACHA20_POLY1305 . Prior to TLS selecting a cipher suite, AES header protection is used, matching the AEAD_AES_128_GCM packet protection.

5.2 Header Protection Sample

The header protection algorithm uses both the header protection key and a sample of the ciphertext from the packet Payload field.

The same number of bytes are always sampled, but an allowance needs to be made for the removal of protection by a receiving endpoint, which will not know the length of the Packet Number field. The sample of ciphertext is taken starting from an offset of 4 bytes after the start of the Packet Number field. That is, in sampling packet ciphertext for header protection, the Packet Number field is assumed to be 4 bytes long (its maximum possible encoded length).

An endpoint MUST discard packets that are not long enough to contain a complete sample.

To ensure that sufficient data is available for sampling, packets are padded so that the combined lengths of the encoded packet number and protected payload is at least 4 bytes longer than the sample required for header protection. The cipher suites defined in [RFC 8446] – other than TLS_AES_128_CCM_8_SHA256,

```
Initial Packet {
    Header Form (1) = 1,
    Fixed Bit (1) = 1,
    Long Packet Type (2) = 0,
    Reserved Bits (2),         # Protected
    Packet Number Length (2),  # Protected
    Version (32),
    DCID Len (8),
    Destination Connection ID (0..160),
    SCID Len (8),
    Source Connection ID (0..160),
    Token Length (i),
    Token (..),
    Length (i),
    Packet Number (8..32),     # Protected
    Protected Payload (0..24), # Skipped Part
    Protected Payload (128),   # Sampled Part
    Protected Payload (..)     # Remainder
}
```

Figure 1: *Header Protection and Ciphertext Sample*

for which a header protection scheme is not defined in this document – have 16-byte expansions and 16-byte header protection samples. This results in needing at least 3 bytes of frames in the unprotected payload if the packet number is encoded on a single byte, or 2 bytes of frames for a 2-byte packet number encoding.

The sampled ciphertext can be determined by the following pseudocode:

```
# pn_offset is the start of the Packet Number field.
sample_offset = pn_offset + 4

sample = packet[sample_offset..sample_offset+sample_length]
```

Where the packet number offset of a short header packet can be calculated as:

```
pn_offset = 1 + len(connection_id)
```

And the packet number offset of a long header packet can be calculated as:

```
pn_offset = 7 + len(destination_connection_id) +
    len(source_connection_id) +
```

```
1-RTT Packet {
    Header Form (1) = 0,
    Fixed Bit (1) = 1,
    Spin Bit (1),
    Reserved Bits (2),         # Protected
    Key Phase (1),             # Protected
    Packet Number Length (2),  # Protected
    Destination Connection ID (0..160),
    Packet Number (8..32),     # Protected
    Protected Payload (0..24), # Skipped Part
    Protected Payload (128),   # Sampled Part
    Protected Payload (..),    # Remainder
}
```

Figure 2: *Header Protection and Ciphertext Sample*

```
    len(payload_length)
if packet_type == Initial:
    pn_offset += len(token_length) +
         len(token)
```

For example, for a packet with a short header, an 8-byte connection ID, and protected with AEAD_AES_128_GCM, the sample takes bytes 13 to 28 inclusive (using zero-based indexing).

Multiple QUIC packets might be included in the same UDP datagram. Each packet is handled separately.

5.3 AES-Based Header Protection

This section defines the packet protection algorithm for AEAD_AES_128_GCM, AEAD_AES_128_CCM, and AEAD_AES_256_GCM. AEAD_AES_128_GCM and AEAD_AES_128_CCM use 128-bit AES in Electronic Codebook (ECB) mode. AEAD_AES_256_GCM uses 256-bit AES in ECB mode.

This algorithm samples 16 bytes from the packet ciphertext. This value is used as the input to AES-ECB. In pseudocode, the header protection function is defined as:

```
header_protection(hp_key, sample):
    mask = AES-ECB(hp_key, sample)
```

5.4 ChaCha20-Based Header Protection

When AEAD_CHACHA20_POLY1305 is in use, header protection uses the raw ChaCha20 function as defined in Section 2.4 of [RFC 8439]. This uses a 256-bit key and 16 bytes sampled from the packet protection output.

The first 4 bytes of the sampled ciphertext are the block counter. A ChaCha20 implementation could take a 32-bit integer in place of a byte sequence, in which case, the byte sequence is interpreted as a little-endian value.

The remaining 12 bytes are used as the nonce. A ChaCha20 implementation might take an array of three 32-bit integers in place of a byte sequence, in which case, the nonce bytes are interpreted as a sequence of 32-bit little-endian integers.

The encryption mask is produced by invoking ChaCha20 to protect 5 zero bytes. In pseudocode, the header protection function is defined as:

```
header_protection(hp_key, sample):
    counter = sample[0..3]
    nonce = sample[4..15]
    mask = ChaCha20(hp_key, counter, nonce, {0,0,0,0,0})
```

6 Receiving Protected Packet

Once an endpoint successfully receives a packet with a given packet number, it MUST discard all packets in the same packet number space with higher packet numbers if they cannot be successfully unprotected with either the same key, or – if there is a key update – a subsequent packet protection key. Similarly, a packet that appears to trigger a key update but cannot be unprotected successfully MUST be discarded.

Failure to unprotect a packet does not necessarily indicate the existence of a protocol error in a peer or an attack. The truncated packet number encoding used in QUIC can cause packet numbers to be decoded incorrectly if they are delayed significantly.

Due to reordering and loss, protected packets might be received by an endpoint before the final TLS handshake messages are received. A client will be unable to decrypt 1-RTT packets from the server, whereas a server will be able to decrypt 1-RTT packets from the client. Endpoints in either role MUST NOT decrypt 1-RTT packets from their peer prior to completing the handshake.

Even though 1-RTT keys are available to a server after receiving the first handshake messages from a client, it is missing assurances on the client state:

- The client is not authenticated, unless the server has chosen to use a pre-shared key and validated the client's pre-shared key binder.
- The client has not demonstrated liveness, unless the server has validated the client's address with a Retry packet or other means.
- Any received 0-RTT data that the server responds to might be due to a replay attack.

Therefore, the server's use of 1-RTT keys before the handshake is complete is limited to sending data. A server MUST NOT process incoming 1-RTT protected packets before the TLS handshake is complete. Because sending acknowledgments indicates that all frames in a packet have been processed, a server cannot send acknowledgments for 1-RTT packets until the TLS handshake is complete. Received packets protected with 1-RTT keys MAY be stored and later decrypted and used once the handshake is complete.

TLS implementations might provide all 1-RTT secrets prior to handshake completion. Even where QUIC implementations have 1-RTT read keys, those keys are not to be used prior to completing the handshake.

The requirement for the server to wait for the client Finished message creates a dependency on that message being delivered. A client can avoid the potential for head-of-line blocking that this implies by sending its 1-RTT packets coalesced with a Handshake packet containing a copy of the CRYPTO frame that carries the Finished message, until one of the Handshake packets is acknowledged. This enables immediate server processing for those packets.

A server could receive packets protected with 0-RTT keys prior to receiving a TLS ClientHello. The server MAY retain these packets for later decryption in anticipation of receiving a ClientHello.

A client generally receives 1-RTT keys at the same time as the handshake completes. Even if it has 1-RTT secrets, a client MUST NOT process incoming 1-RTT protected packets before the TLS handshake is complete.